Лабораторна робота №2

Функціональне програмування

Багатопотоковий архіватор Хаффмана

Левківський Євгеній Сергійович

ІПС-41

1. Мета роботи

Реалізувати багатопотоковий застосунок на Haskell, використовуючи GHC з ключем -threaded. Дослідити швидкодію паралельного стиснення множини файлів алгоритмом Хаффмана на багатопроцесорній системі.

2. Постановка завдання

Обрано перше завдання: стиснути множину файлів в один архів алгоритмом Хаффмана та показати ефективність за часом. Додатково реалізовано розпакування, перегляд вмісту та контроль цілісності, щоб перевірити правильність стиснення.

Код розділено на Huffman.hs — алгоритм, Archive.hs — формат і файлові операції, WorkerPool.hs — пул потоків, Main.hs — команди та експеримент. Стиснення окремих файлів незалежне; результати записуються в один архів HFAR v1 у початковому порядку.

3. Алгоритм Хаффмана

Спочатку підраховуються частоти всіх байтів вхідного файла. Для кожного байта створюється листок дерева з вагою, що дорівнює частоті. Два вузли з найменшими вагами зливаються; процес повторюється до отримання кореня. Перехід ліворуч додає біт 0, праворуч — 1. У результаті отримуються префіксні коди, які дозволяють однозначне відновлення.

Алфавіт складається максимум із 256 байтів. Вузли впорядковуються за частотою та найменшим символом піддерева, що гарантує однакове дерево під час стиснення й відновлення. Порожній файл має порожнє дерево. Якщо є лише один унікальний байт, кількість повторень відома з початкового розміру й бітовий потік порожній.

Кодування працює з двійковими ByteString, тому не залежить від кодування вмісту файлів. Назви зберігаються як UTF-8. Коди представлені Integer, що не обмежує глибину дерева машинним словом. Залишок останнього байта доповнюється нулями. Для контролю відновленого вмісту обчислюється CRC32 IEEE.

4. Паралельна реалізація

parallelMap створює фіксовану кількість працівників, не більшу за кількість файлів. Канал Chan містить індексовані завдання. Кожен працівник стискає один файл, повністю обчислює результат через evaluate і force та передає його до каналу результатів. Наприкінці результати сортуються за початковим індексом.

Винятки працівників передаються головному потоку. bracket гарантує завершення працівників після успіху або помилки. Ключ -threaded підключає багатопотокове середовище виконання, а параметр +RTS -N4 надає чотири capabilities для виконання Haskell-коду на різних ядрах.

```
lab2 pack 4 demo.hfar file1.txt file2.txt +RTS -N4 -RTS
lab2 list demo.hfar
lab2 unpack demo.hfar restored
```

Формат зберігає заголовок HFAR, версію, кількість файлів, а для кожного файла — ім’я, початковий розмір, частотну таблицю, число бітів, CRC32 та стиснені байти. Повну специфікацію наведено в lab2/README.md.

5. Методика вимірювання

Експеримент виконано на Windows x64, AMD Ryzen 5 5600H, 12 логічних процесорів, GHC 9.6.7. Прапорці компіляції: -O2 -Wall -threaded -rtsopts. Вхідний набір — 8 файлів по 1 МіБ, разом 8 388 608 байтів; файли створює scripts/benchmark.mjs повторенням навчального тексту з третьої роботи.

Вхідні дані завантажуються й повністю обчислюються до початку вимірювань. Один прогрівальний запуск не враховується. Для 1, 2 та 4 працівників виконано по три вимірювання монотонним годинником getMonotonicTimeNSec; до кожного запуску виконується performGC. Обрано медіану. Кількість capabilities налаштовується відповідно до кількості працівників.

Таймер охоплює пул потоків і саме стиснення. Читання з диска, серіалізація архіву, запис та декодування в цей час не входять. Після кожного вимірювання перевіряється повне відновлення файлів. Таким чином,

наведені числа характеризують обчислювальну частину архіватора.

Прискорення визначено як S(k) = T(1) / T(k), ефективність — E(k) = S(k) / k.

6. Отримані результати

Працівники	Медіана часу, с	Прискорення S	Ефективність E
1	0.670740	1.0000	1.0000
2	0.399344	1.6796	0.8398
4	0.228518	2.9352	0.7338

У дослідженому наборі чотири працівники зменшили час стиснення приблизно у 2,94 раза. Прискорення менше числа працівників через синхронізацію, керування потоками, роботу збирача сміття та спільний доступ до пам’яті. Результат стосується цього набору даних і обладнання; для інших файлів та навантаження час змінюється.

Фактичний вивід збережено у verification/benchmark.txt, числові дані — в lab2/benchmark.csv, параметри системи — у verification/environment.json. Повторення експерименту: node scripts/benchmark.mjs.

7. Перевірка правильності й обмеження

Автоматичні тести охоплюють порожні дані, один символ, всі 256 байтів, повторювані й псевдовипадкові дані та український текст. Перевірено серіалізацію архіву, неправильну сигнатуру, обрізаний вміст, дублікати імен, пошкоджену контрольну суму й неправильний бітовий потік. Результати паралельного й послідовного стиснення збігаються.

Інтеграційний запуск створив архіви в 1 та 4 потоки; файли архівів побайтно ідентичні. Після розпакування всі файли, включно з Unicode-іменем, збігаються з оригіналами. Повторний запис у наявні файли та нуль потоків відхиляються. Пошкоджений архів не створив вихідної директорії.

Максимум формату в реалізації: 10000 файлів, 512 МіБ на файл. Дані обробляються в пам’яті; доступна RAM обмежує розмір набору. Імена

небезпечних шляхів і зарезервовані Windows-імена відхиляються. Архіватор розпаралелює файли, тому для одного файла прискорення між файлами немає. Невеликі файли можуть збільшуватись через службову інформацію.

8. Висновок

Реалізовано багатопотокове стиснення множини файлів алгоритмом Хаффмана в один архів з можливістю відновлення. Підтверджено правильність побайтовими порівняннями та контрольними сумами. Експеримент показав прискорення 2,9352 на чотирьох працівниках для восьми файлів загальним розміром 8 МіБ.

9. Джерела

GHC User’s Guide, паралельне виконання та RTS: https://downloads.haskell.org/ghc/9.6.7/docs/users_guide/

Бібліотеки Haskell base, bytestring, binary, containers: https://hackage.haskell.org/